

Numerische Methoden – Übungsaufgaben

Vollständig aus den Vorlesungsaufschriften von Prof. Dr.-Ing. Mark Schutera extrahiert · ins Deutsche übersetzt

Diese Sammlung enthält *alle* im Skript gestellten Aufgaben – benannte Übungen, Handrechnungen, Code-/Maschinen-Aufgaben, Margin-Aufgaben, Denkanstöße und Selbstreflexionsfragen – über alle sechs Kapitel. Ausgearbeitete Musterlösungen sind *bewusst nicht* enthalten. Lösungswege: siehe separates Dokument `schutera_loesungen`.

Kapitel 1 – Einstieg in die Numerischen Methoden

1.1 – Größeres Gleichungssystem [Handrechnung]

Mit der Größe des Systems steigt der Rechenaufwand. Lösen Sie nun dieses größere System aus drei Gleichungen mit drei Unbekannten von Hand:

$$\begin{pmatrix} 2 & 1 & 3 \\ 1 & 4 & 2 \\ 3 & 2 & 5 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = \begin{pmatrix} 14 \\ 20 \\ 32 \end{pmatrix}$$

1.2 – Grid-Search [Handrechnung]

Führen Sie eine Grid-Search für w und b mit Schrittweite 2,5 über den Bereich $w, b \in \{0; 2,5; 5; 7,5; 10\}$ durch. Berechnen Sie für jede Kombination den Fehler $\text{error}_1 = |2w + b - 11|$, $\text{error}_2 = |5w + b - 13|$. Werten Sie alle Kombinationen aus und wählen Sie das Paar (w, b) mit dem kleinsten akkumulierten Fehler.

1.3 – Enter the machine [Code]

Viele numerische Verfahren sind für die Umsetzung am Computer gedacht. Sie erhalten ein vorgehostetes Jupyter-Notebook mit einer Python-Vorlage für diese Übung. Nutzen Sie die Selbstreflexionsfragen unten, um sich beim Erkunden und Experimentieren mit der Implementierung leiten zu lassen.

Denkanstoß. Fällt Ihnen eine einfache Möglichkeit ein, das Verhältnis von Genauigkeit zu Rechenaufwand unseres Grid-Search-Beispiels von oben zu verbessern?

Selbstreflexionsfragen

- Warum ist die Wahl der Schrittweite h beim Diskretisieren einer stetigen Funktion wichtig, und wie beeinflusst sie Genauigkeit und Rechenzeit der numerischen Lösung?
- Wie beeinflusst die Wahl des Intervalls $[a, b]$ die Ergebnisse der Diskretisierung und die Lage der numerisch gefundenen Extrema?
- Was sind die wesentlichen Unterschiede zwischen einer stetigen Funktion und ihrer diskretisierten Version, und welche Implikationen ergeben sich für das numerische Lösen mathematischer Probleme?

Kapitel 2 – Gleitkomma-Arithmetik

2.1 – Rundungsfehler [Handrechnung]

Bekommen wir an einem einfachen Beispiel ein Gefühl für den Rundungsfehler. Betrachten Sie die Division von 10 durch 3. Schreiben Sie die Dezimalentwicklung aus.

2.2 – Maschinenepsilon (hands on) [Code]

Bevor Sie an den Rechner gehen: Überlegen Sie, wie Sie das Maschinenepsilon eines beliebigen Systems für ein bestimmtes Gleitkommaformat per Programm bestimmen würden. Rufen Sie sich die Definition des Maschinenepsilons in Erinnerung. Schreiben Sie Ihre Überlegungen als Pseudocode auf. Was würde ein solches Programm auf einem Festkomma- gegenüber einem Gleitkommasystem zeigen? Schreiben Sie Ihre Gedanken auf. Probieren Sie dann verschiedene Datentypen am Rechner im Code aus. Notieren Sie Ihre Beobachtungen und reflektieren Sie sie. Wenn Sie einen Taschenrechner bauen würden – welchen Datentyp würden Sie wählen und warum?

2.3 – Auslöschung (loss of significance) [Code]

Überlegen Sie nun, wie Sie Auslöschung an Ihrem Rechner demonstrieren würden. Schreiben Sie Ihre Überlegungen als Pseudocode auf, bevor Sie weiterlesen.

2.4 – Argumentation [Konzept]

Überlegen Sie, wie das Berechnen von $f(\epsilon) = \frac{1}{a + \epsilon - b}$ für kleines ϵ und $a = b$ dies leisten würde. Was erwarten Sie zu sehen, wenn ϵ sehr klein ist?

Margin: Was passiert für $a > b$? Denken Sie in signifikanten Stellen.

Denkanstoß. Fallen Ihnen Metriken für numerische Verfahren ein, basierend auf den Approximationen und Fehlern, die wir besprochen haben?

Selbstreflexionsfragen

- Wie ist die Gleitkommadarstellung aufgebaut, und was sind ihre Komponenten?
- Was ist das Maschinenepsilon, und wie hängt es mit der numerischen Genauigkeit zusammen?
- Wie wirken sich diese Konzepte in der Praxis auf numerische Berechnungen aus?

Kapitel 3 – Fehleranalyse

3.1 – Minimum der Sinusfunktion [Handrechnung]

Bestimmen Sie das Minimum der Sinusfunktion auf dem Intervall $[-3, 1]$ noch einmal. Diskretisieren Sie die Funktion erneut mit zwei verschiedenen Schrittweiten $h = 1$ und $h = 0,2$. Nehmen Sie weiter an, dass die Eingabedaten x durch Messfehler um $\tilde{x} = x \pm 0,25$ gestört sind und eine Genauigkeit von zwei Dezimalstellen besitzen. Analysieren Sie nun Kondition, Stabilität, Konsistenz und Konvergenz des zur Minimumsuche verwendeten numerischen Verfahrens in beiden Fällen. Quantifizieren Sie die Eigenschaften mithilfe der im vorigen Abschnitt angegebenen Formeln.

3.2 – Stabilität und Kondition [Beweis]

Margin: Die Stabilität entspricht der Kondition des Systems für $h \rightarrow 0$. Zeigen Sie das.

3.3 – Compute-getriebene Fehleranalyse (hands on) [Code]

Brute-forcen wir uns durch die Fehleranalyse beim Lösen des Zwei-Gleichungs-Systems aus Kapitel 1. Sie bestimmen und optimieren Kondition, Stabilität, Konsistenz und Konvergenz der numerischen Lösung, indem Sie das numerische Verfahren optimieren.

Selbstreflexionsfragen

- Wie gut konditioniert ist das Zwei-Gleichungs-System verglichen mit der numerischen Lösung des Minimums der Sinusfunktion?
- Konvergiert die Stabilität beim Zwei-Gleichungs-System mit kleineren Schrittweiten? Wogegen?
- Wie wirken sich Störungen der Eingabedaten auf die numerische Lösung des Zwei-Gleichungs-Systems aus? Gibt es ein Zusammenspiel mit der Schrittweite?
- Vergleichen Sie Konvergenz mit Stabilität und Konsistenz. Was beobachten Sie? Können Sie Ihre Beobachtungen in Begriffen von Bias und Varianz ausdrücken?
- Beobachten Sie für immer kleinere Schrittweiten eine Grenze der Genauigkeit der numerischen Lösung? Wenn ja, warum?
- Welches weitere Problem beobachten Sie beim Verfeinern der Schrittweite?

Denkanstoß. Bisher haben wir Brute-Force-Lösungen verwendet. Fällt Ihnen eine Möglichkeit ein, Brute-Force-Methoden zu verfeinern, um mit weniger Aufwand bessere Ergebnisse zu erzielen? Nutzen Sie den Code dieser Vorlesung, um Ihre Idee zu entwerfen und zu testen.

Kapitel 4 – Newton-Verfahren

4.0 – Denksportaufgabe: Versagensmodi [Schaubild]

Das Newton-Verfahren kann auf verschiedene Weisen versagen. Als Denksportaufgabe ist es eine schöne Übung, die Versagensmodi in diesem Schaubild zu visualisieren (Abb.: $f(x) = x^3 - x$).

4.1 – Newton von Hand [Handrechnung]

Finden Sie die Nullstelle von $f(x) = x^3 - x = 0$, indem Sie das Newton-Verfahren geometrisch und anschließend von Hand anwenden. Die Nullstellen liegen bei $x = -1, 0, 1$. Suchen wir die Nullstelle bei $x = 1$, ausgehend von $x_0 = 1,4$.

4.2 – Sekantenverfahren von Hand [Handrechnung]

Wenden Sie das Sekantenverfahren auf dasselbe Problem mit $x_0 = 1,2$, $x_1 = 1,4$ an. Wieder zuerst geometrisch, dann von Hand.

4.3 – Geometrische Fehlersuche [Handrechnung]

Finden Sie Startpunkte x_0 für verschiedene Versagensmodi: einen, bei dem das Newton-Verfahren wegen verschwindender Ableitung versagt, einen, bei dem es zu einer Nicht-Ziel-Nullstelle konvergiert, und einen, bei dem es zwischen Punkten oszilliert.

4.4 – Enter the machine [Code]

Betrachten wir Konvergenz und Konvergenzraten von Grid-Search, Newton- und Sekantenverfahren in Python. Experimentieren Sie dann mit verschiedenen Startpunkten und beobachten Sie die Versagensmodi in der Praxis. Versuchen Sie, die Startpunkte zuerst geometrisch zu finden, dann analytisch, bevor Sie sie schließlich im Code verifizieren.

Selbstreflexionsfragen

- Was ist der Hauptvorteil des Newton-Verfahrens gegenüber der Grid-Search?
- Warum genügt eine lineare Approximation zum Finden von Nullstellen? Was sagt uns die Taylor-Entwicklung über diese Wahl?
- Warum ist quadratische Konvergenz so mächtig? Wie viele Iterationen bräuchte Newton, um von Fehler 10^{-1} auf 10^{-16} zu kommen?
- In welchen Situationen würden Sie das Sekantenverfahren dem Newton vorziehen? Warum nicht in anderen?

Denkanstoß. Wie können wir sicherstellen, dass wir die globale Lösung finden und nicht nur eine lokale?

Kapitel 5 – Globale Optimierung

5.0 – Newton auf Shekel, weiterer Startpunkt [Code]

Wenden wir das Newton-Verfahren auf unsere 1D-Shekel-Funktion von verschiedenen Startpunkten an und beobachten, wohin das Verfahren konvergiert. Wir zeigen es für einen Startpunkt; probieren Sie selbst einen weiteren, und dann diskutieren wir das allgemeine Verhalten.

Margin: Wie würden Sie die Update-Regel abändern, um stattdessen Maxima zu finden?

5.1 – Random Restarts [Handrechnung]

Angenommen, Sie optimieren eine Funktion mit 5 lokalen Minima, und das Einzugsgebiet des globalen Minimums deckt 15% des Suchraums ab ($p = 0,15$). Wie groß ist die Wahrscheinlichkeit, das globale Minimum mit $K = 10$ Random Restarts zu finden?

Hinweis: Nutzen Sie $P = 1 - (1 - p)^K$ und lösen Sie nach K auf: $K = \frac{\ln(1 - P)}{\ln(1 - p)}$.

(Folge) Wie viele Restarts brauchen wir für 99%? Berechnen Sie nun K für $p = 0,05$ und $p = 0,01$ beim selben 99%-Ziel.

5.2 – Eigene 1D-Shekel-Funktion [Code]

An Ihrem Rechner entwerfen Sie zunächst Ihre eigene 1D-Shekel-Funktion. Halten Sie sie anfangs einfach, machen Sie sie später komplexer. Machen Sie sich dann zuerst mit den bisher gelernten lokalen Optimierungsverfahren vertraut und wenden Sie sie auf Ihre Funktion an. Zeigen Sie, wo das Newton-Verfahren (erneut) versagt. Untersuchen Sie dann verschiedene x_0 und sehen Sie, wie sich das Konvergenzverhalten ändert. Denken Sie wieder über die Loss-Landschaft und Basin-Grenzen nach.

5.3 – Die Suche zu Minima lenken [Konzept + Code]

Erinnern Sie sich an den $|f''|$ -Trick, der das Vorzeichen der Krümmung im Nenner umkehrt und den Schritt zwingt, stets bergab zu gehen. Überlegen Sie, was das geometrisch für die Tangentenkonstruktion bedeutet. Probieren Sie es dann im Code aus.

5.4 – Einzugsgebiet (Basin of Attraction) [Schaubild]

Betrachten Sie erneut die Plots der Shekel-Funktion, markieren Sie die Einzugsgebiete jedes lokalen Minimums und schätzen Sie ihre relativen Größen ab. Welches ist das globale Minimum? Wie hängt das mit der Wahrscheinlichkeit zusammen, es per Random Restarts zu finden?

5.5 – Lokalen Minima entkommen [Code]

Setzen Sie Random Restarts und Basin Hopping auf Ihrer eigenen 1D-Shekel-Funktion ein und vergleichen Sie, wie viele Funktionsiterationen jede Methode bis zum globalen Minimum braucht. Wie anfällig sind sie, in lokalen Minima hängen-zubleiben? Wie beeinflusst die Wahl der Schrittweitenverteilung die Performance des Basin Hopping?

5.6 – Nicht-konvexe Becken [Konzept]

Shekel ist im Allgemeinen nicht-konvex, die einzelnen Becken jedoch schon; notieren Sie eine Funktion, bei der die Einzugsgebiete nicht so leicht abzugrenzen sind.

5.7 – Adaptive Schrittweite [Konzept + Code]

Fällt Ihnen ein geschickter Weg ein, die Schrittweitenverteilung während der Optimierung automatisch anzupassen? Was wäre eine gute Heuristik? Wie könnten Sie katastrophale Sprünge verhindern? Implementieren Sie es.

5.8 – Noise and landscape engineering [Konzept]

Die Loss-Landschaft ist nicht statisch, sondern soll geformt und gestaltet werden. Indem man die Loss-Landschaft auf Mini-Batches (unterschiedliche Auswahl von Punkten) approximiert ...

5.9 – Code-Übung: Restarts vs. Basin Hopping [Code]

Implementieren Sie Random Restarts und Basin Hopping auf der 1D-Shekel-Funktion. Vergleichen Sie, wie viele Funktionsauswertungen jede Methode braucht, um das globale Minimum bei $x \approx 4$ zu finden.

5.10 – Einzugsgebiete kartieren [Handrechnung]

Kartieren wir schließlich die Einzugsgebiete. Betrachten Sie die 1D-Shekel-Funktion und nehmen Sie an, ein lokaler Optimierer konvergiere stets zum nächstgelegenen Foxhole.

- Skizzieren Sie (grob) die Einzugsgebiete für die drei Foxholes bei $x = 0, 4, 7$. Wo liegen die Basin-Grenzen?
- Wenn Basin Hopping einen Sprung $\delta \sim \text{Uniform}(-3, 3)$ ausgehend vom lokalen Minimum $x^\circ = 7$ verwendet, wie groß ist die Wahrscheinlichkeit, dass ein einzelner Sprung im Becken des globalen Minimums landet?
- Warum könnte Basin Hopping das globale Minimum hier schneller finden als Random Restarts?

Selbstreflexionsfragen

- Warum versagt lokale Optimierung auf nicht-konvexen Landschaften wie Shekels Foxholes?
- Wie skaliert der erforderliche Aufwand mit der Größe des globalen Beckens bei Random Restarts?
- Wie unterscheidet sich Basin Hopping von Random Restarts?
- Was sagt uns $f''(x_n)$ über die Loss-Landschaft, was charakterisiert es, und wie können wir es nutzbar machen?
- Wie hilft das Approximieren der Loss-Landschaft mit verrauschten Schätzungen (Mini-Batches), lokalen Optima zu entkommen?
- Wählen Sie x_0 so, dass es im Becken eines lokalen Minimums liegt; finden Sie nun verschiedene Wege, mit denen Ihr Optimierungsverfahren entkommen kann – wie wirksam ist welcher?

Denkanstoß. Was, wenn wir nun einer hochfrequenten Version von Shekels Foxholes gegenüberstehen? Welches Problem tritt dabei auf?

Kapitel 6 – Numerische Integration

6.1 – Exakter Wert [Handrechnung]

Bleiben wir bei unserem Beispiel aus dem Methodenabschnitt und berechnen das Integral von $\sin(x)$ von -2 bis -1 mit verschiedenen Methoden. Um den Fehler jeder Methode quantifizieren zu können, brauchen wir den exakten Wert. Berechnen wir also den exakten Wert des Integrals.

6.2 – Mittelpunktsregel von Hand [Handrechnung]

Berechnen Sie $\int_{-2}^{-1} \sin(x) dx$ mit der Mittelpunktsregel mit $n = 1$ Intervall.

6.3 – Mittelpunktsregel mit mehr Intervallen [Handrechnung]

Berechnen Sie nun die Mittelpunktsregel mit $n = 2$ Intervallen, d. h. wir haben zwei Mittelpunkte bei $-1,75$ und $-1,25$.

Margin: Weiter lohnt es sich, die Approximation an den Endpunkten des Intervalls zu berechnen und den Fehler zu vergleichen.

6.4 – Trapezregel von Hand [Handrechnung]

Berechnen Sie mit der Trapezregel $\int_{-2}^{-1} \sin(x) dx$ mit $n = 2$ Intervallen. Die Stützstellen sind $-2, -1,5$ und -1 .

6.5 – Simpson-Regel von Hand [Handrechnung]

Berechnen Sie mit der Simpson-Regel $\int_{-2}^{-1} \sin(x) dx$ mit $n = 2$ Intervallen. Die Stützstellen sind $-2, -1,5$ und -1 .

Margin: Probieren Sie mehr Intervalle, sehen Sie, wie der Fehler abnimmt, und prüfen Sie, ob Sie sich mit

zusammengesetzten Simpson- und Trapezregeln auskennen.

6.6 – Monte Carlo von Hand [Handrechnung]

Schätzen Sie $\int_{-2}^{-1} \sin(x) dx$ per Monte Carlo mit $N = 4$ Zufallsstichproben, was etwa dem Rechenaufwand der Trapezmethode entspricht. Nehmen Sie an, die Zufallsstichproben sind: $X_1 = -1,95$, $X_2 = -1,55$, $X_3 = -1,15$, $X_4 = -1,05$.

6.7 – Monte Carlo: N erhöhen [Konzept]

Was passiert, wenn wir N auf 16 erhöhen? ... Erläutern Sie kurz, wie der Fehler des Monte-Carlo-Schätzers dimensionsunabhängig ist.

6.8 – Simpson-Gewichte via Lagrange [Herleitung]

Margin: Probieren Sie es mit ein paar Punkten. Beginnen Sie mit $x = 0$ und sehen Sie, wie die Linearfaktoren ausfallen. (Im Skript wird die Rechnung übersprungen: Leiten Sie die Quadraturgewichte durch Integration der Lagrange-Basispolynome über $[a, b]$ selbst her.)

Margin: Was passiert, wenn Sie eine lineare Funktion mit einer quadratischen approximieren?

Margin: Was, wenn wir über ein Intervall integrieren, das zu destruktiver Interferenz der zugrundeliegenden höherfrequenten Sinuskurve führt? Denken Sie an Wellenlängen.

6.9 – Höhere Dimensionen (on your machine) [Code]

Implementieren Sie Monte-Carlo-Integration für ein d -dimensionales Integral, etwa die Integration einer multivariaten Gauß-Funktion über einen Hyperwürfel. Während Sie d langsam erhöhen, beobachten Sie, wie sich der Fehler für verschiedene Methoden verhält. Beobachten Sie zudem, wie gitterbasierte Quadraturmethoden mit wachsendem d rechnerisch undurchführbar werden. Optional: Sehen Sie, wie ähnliche Effekte in der Optimierung auftreten, indem Sie Gradientenabstieg und Mini-Batch-SGD auf einer hochdimensionalen Funktion implementieren. Achten Sie nur auf die Rechenzeiten.

Selbstreflexionsfragen

- Wie vergleichen sich die Fehler von Mittelpunkts-, Trapez-, Simpson-Regel und Monte Carlo?
- Warum hat die Mittelpunktsregel eine bessere Genauigkeit als die Links- oder Rechts-Endpunkt-Regel?
- Wie verbessern zusammengesetzte Methoden die Genauigkeit, und was ist der Trade-off?
- Warum versagen deterministische Quadraturmethoden in hohen Dimensionen?
- Inwiefern ist die Mittelpunktsregel ein Spezialfall der Monte-Carlo-Schätzung?
- Was ist die Intuition dahinter, dass Monte Carlo in hohen Dimensionen nicht "explodiert"?
- Wann würden Sie die Simpson-Regel gegenüber Monte Carlo verwenden?
- Wie hängt der Mini-Batch-Mittelwert in SGD mit der Monte-Carlo-Schätzung zusammen?
- Wie hilft die Gradientenschätzung in SGD, lokalen Minima zu entkommen?

Denkanstoß. Warum werden hochgradige Quadraturmethoden instabil, wenn sie Polynome niedrigen Grades approximieren?